

Convolution Neural Network

Week 12

D4 Robotics Engineering Study Program

Department of Electrical Engineering

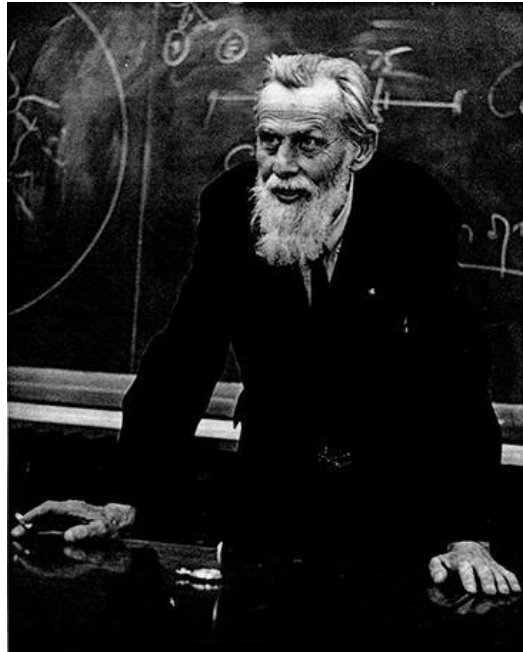
Politeknik Negeri Batam

Machine Learning

Key Milestones in Neural Networks Development

01

Key Milestones in Neural Networks Development



1949

Donald Hebb

Developed a foundational **learning rule**



1958

Frank Rosenblatt

Created the first **Perceptron** model.

1943

McCulloch & Pitts

Input aggregation for computational power, threshold for firing characteristic



Key Milestones in Neural Networks Development



1970

Seppo Linnainmaa

automatic differentiation (AD) of discrete connected networks of nested differentiable functions, corresponding to Backpropagation

1973

Stuart Dreyfus

used backpropagation to adapt parameters of controllers in proportion to error gradients



1974

Paul Werbos

applying backpropagation parameters to artificial neural networks.



Key Milestones in Neural Networks Development



1998

Yann LeCun, Leon Bottou, Yoshua Bengio, Patrick Haffner
deep Convolutional Neural Network for digits recognition

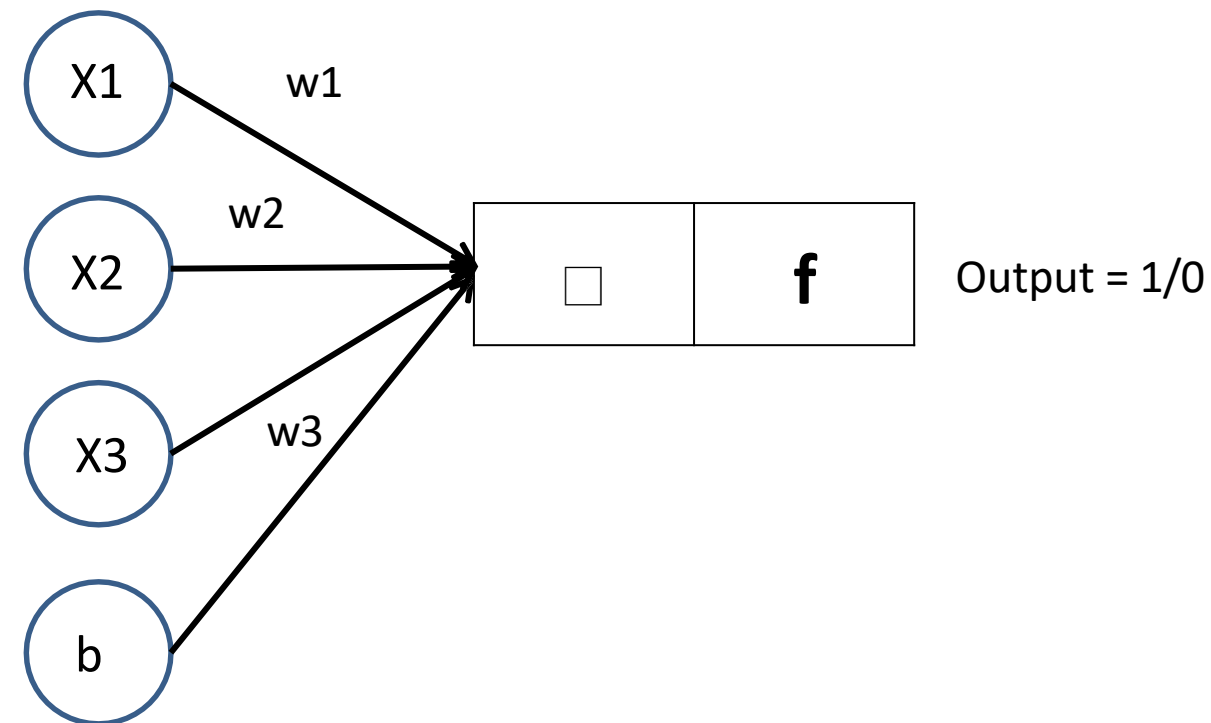


1997

Sepp Hochreiter and Jurgen Schmidhuber
introduced **Long-Short Term Memory (LSTM)** networks, enabling better learning of long-term dependencies.



Simple Neural Network



$$\text{Output} = f(w_1 \times I_1 + w_2 \times I_2 + \text{bias})$$

Inputs: **I1** and **I2** as variables.

The neural network receives two inputs, denoted as **I1** and **I2**, which are essential for the network's operation.

Weights: The role of **w0**, **w1**, **w2**.

Weights (**w0**, **w1**, **w2**) adjust the influence of each input on the output, determining the network's behavior.

Threshold: Understanding the Bias term.

The **Bias** acts as a threshold that influences whether the neuron activates, impacting the output result.

What can perceptron represent

✓ Bisa Diselesaikan:

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

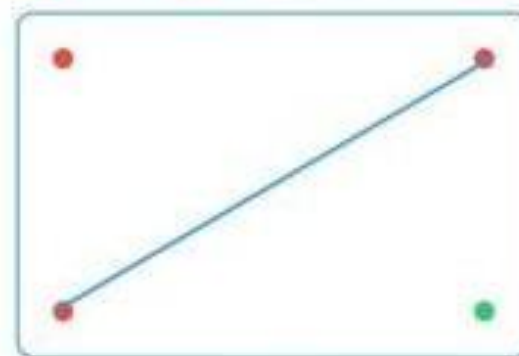
Linearly Separable - Bisa dipisah dengan garis lurus

✗ Tidak Bisa Diselesaikan:

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Non-linearly Separable - Tidak bisa dipisah dengan garis lurus

AND Gate - Linearly Separable



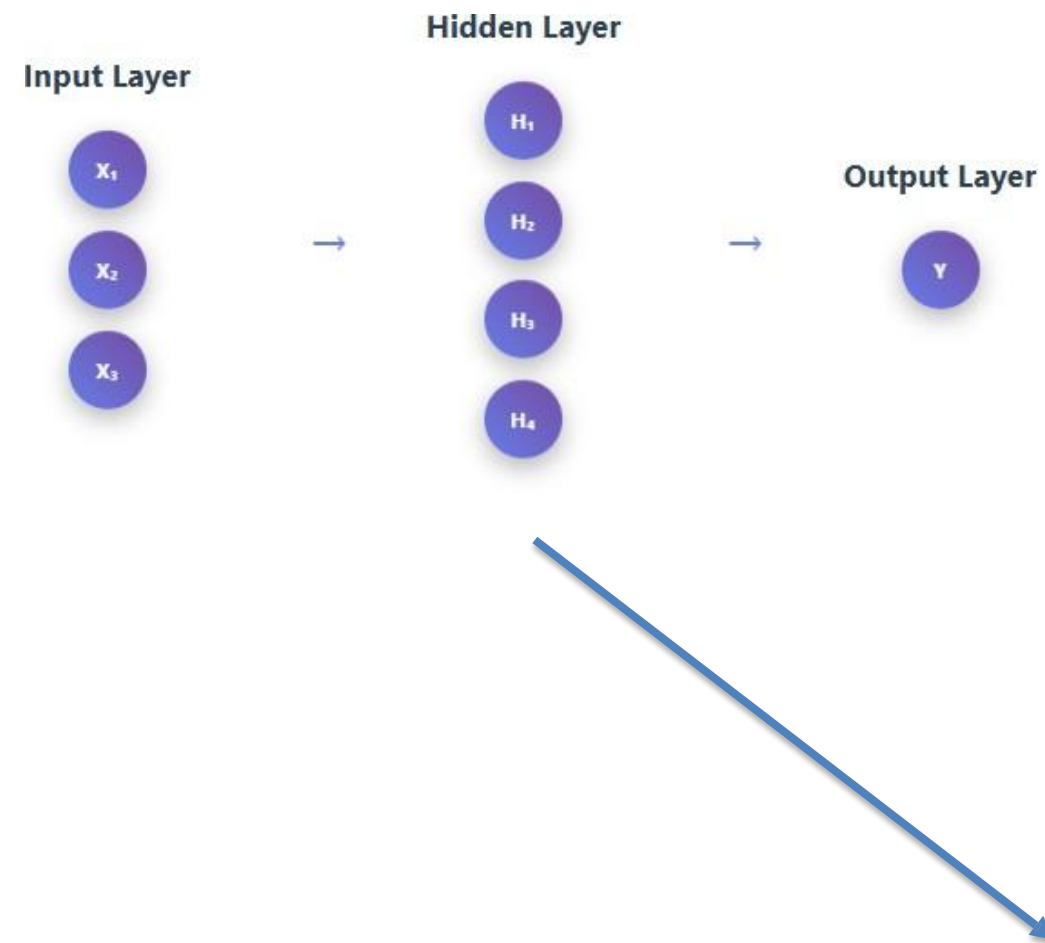
● Output = 0 ● Output = 1
Satu garis bisa memisahkan!

XOR Gate - Non-linearly Separable

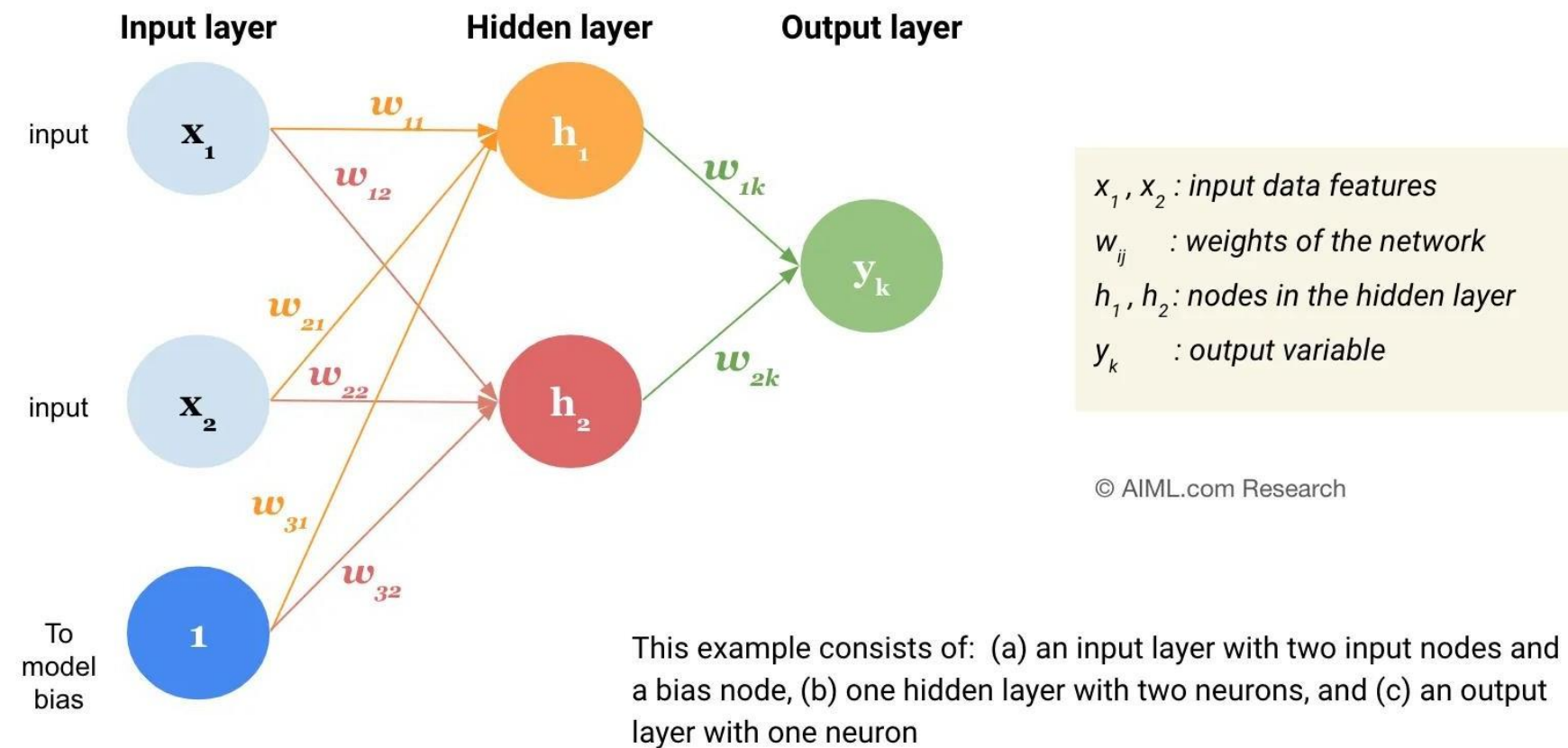


● Output = 0 ● Output = 1
Tidak ada garis yang bisa memisahkan!

Multi Layer Perceptron

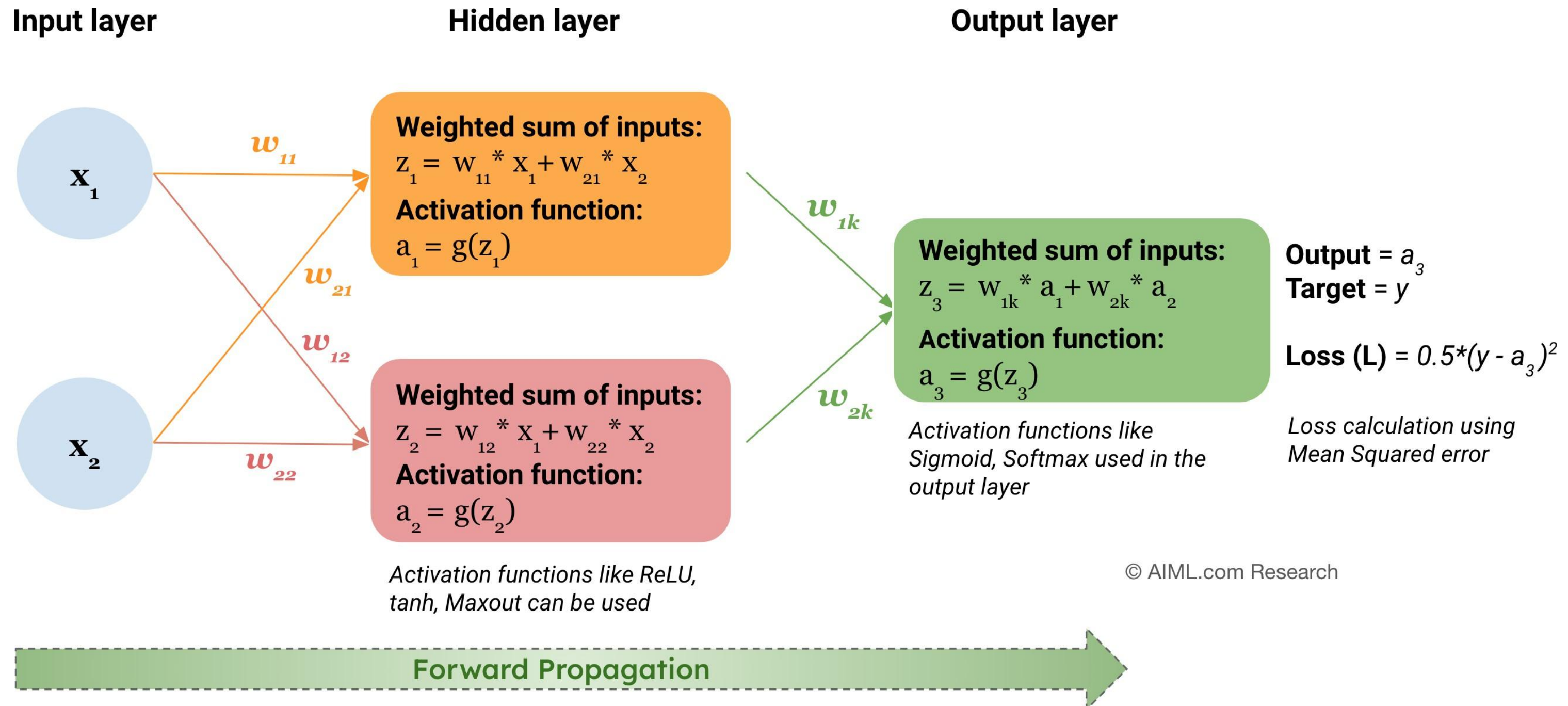


Illustrative example of Multilayer perceptron, a Feedforward neural network



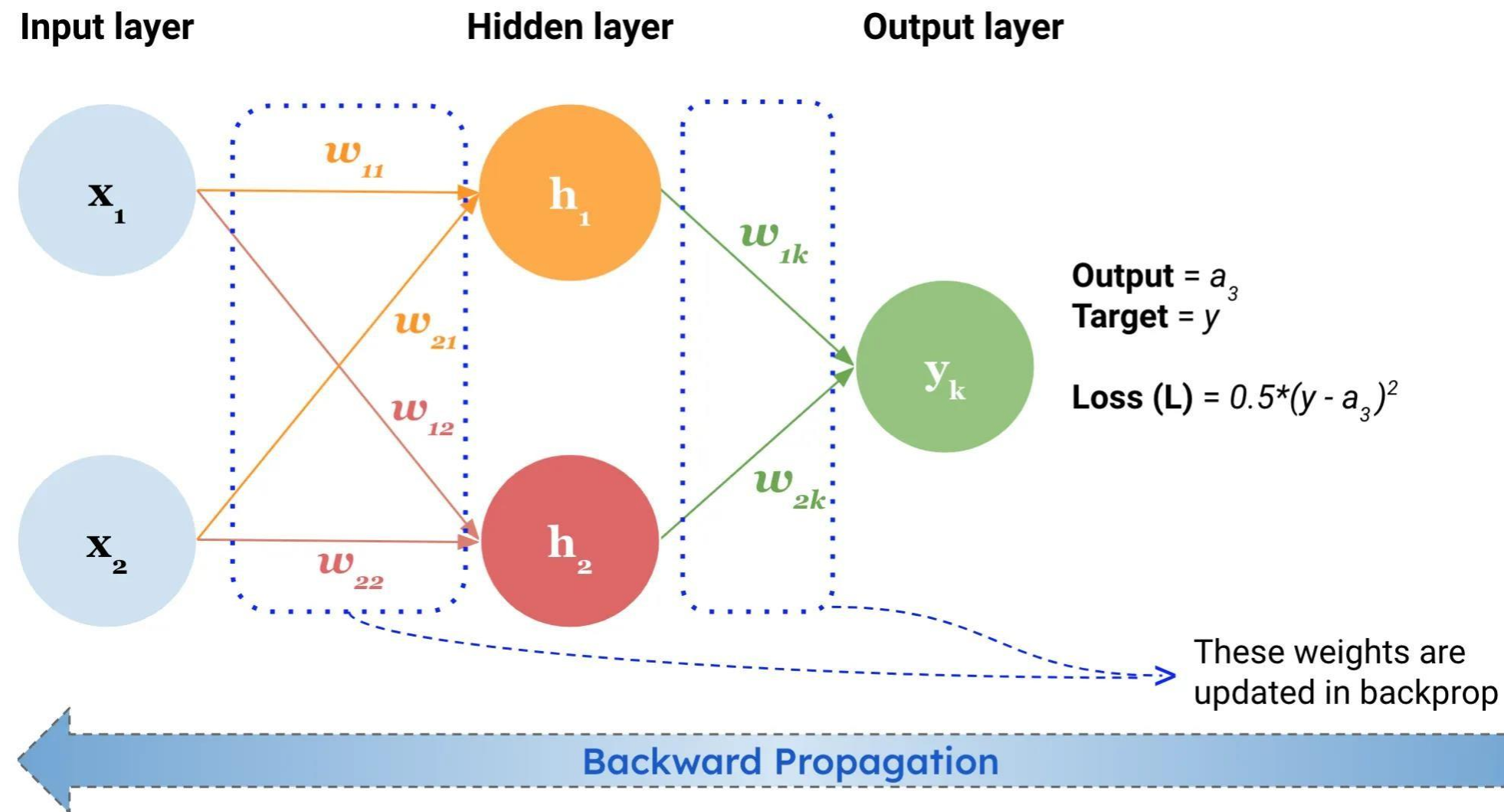
Multi Layer Perceptron

Forward Propagation



Multi Layer Perceptron

Backward Propagation



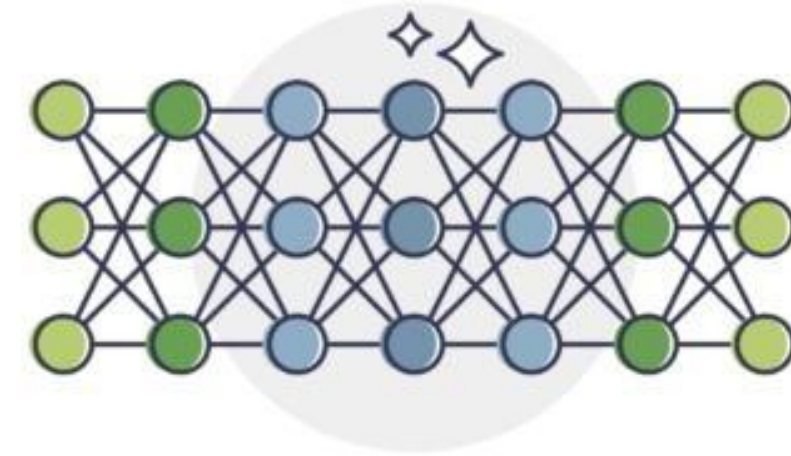
- In stochastic gradient descent, weights (w_{ij}) are updated as below:

$$w_{ij} := w_{ij} + \Delta w_{ij}$$

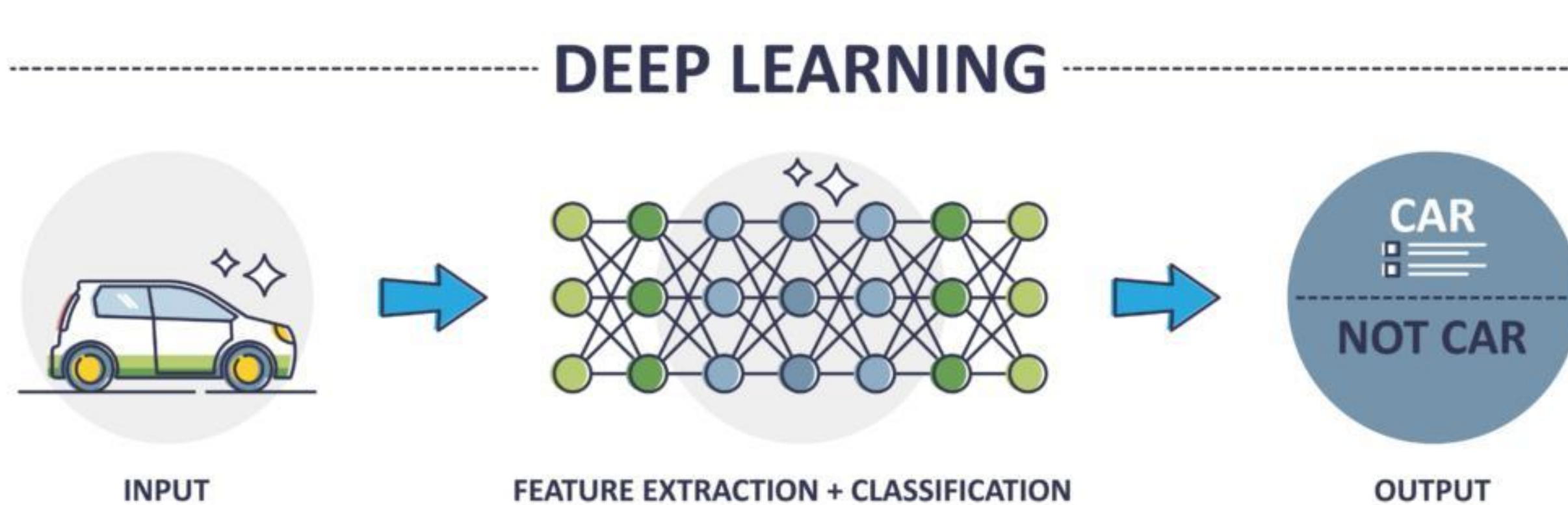
$$\Delta w_{ij} = -\eta * \partial L(w_{ij}) / \partial w_{ij},$$
 where η is the learning rate and $\partial L(w_{ij}) / \partial w_{ij}$ is the gradient of loss w.r.t the model weights
- Intermediate variables calculated during forward prop ($z_1, z_2, z_3, a_1, a_2, a_3$) are used for gradient calculation $\partial L(w_{ij}) / \partial w_{ij}$

Deep Learning

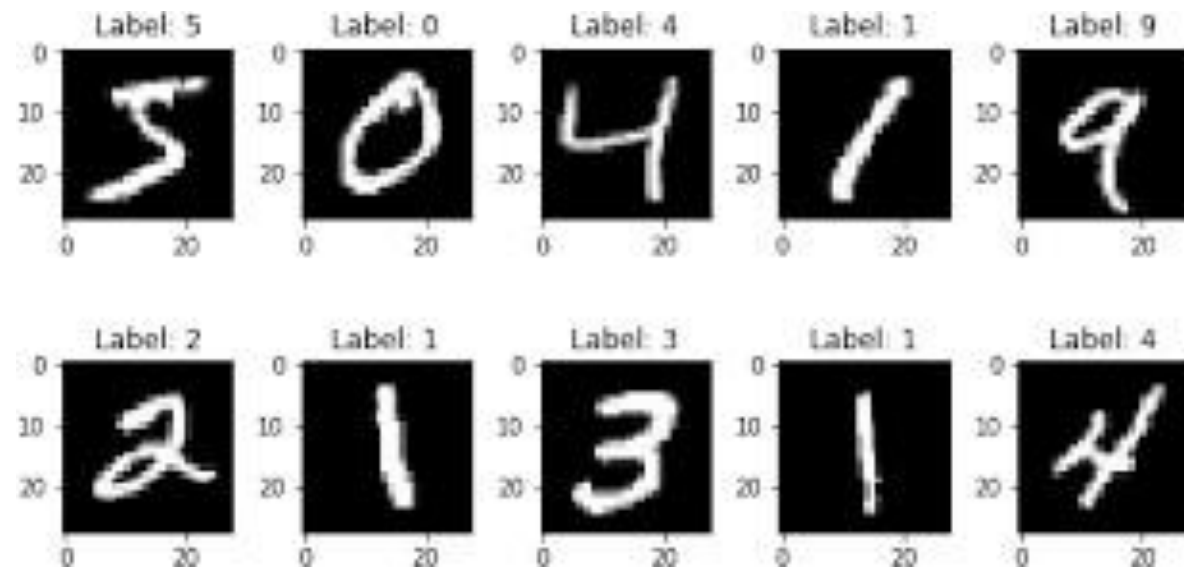
- Jaringan saraf dengan berbagai lapisan.
- Dua proses utama:
 - Pembelajaran Fitur
 - Klasifikasi



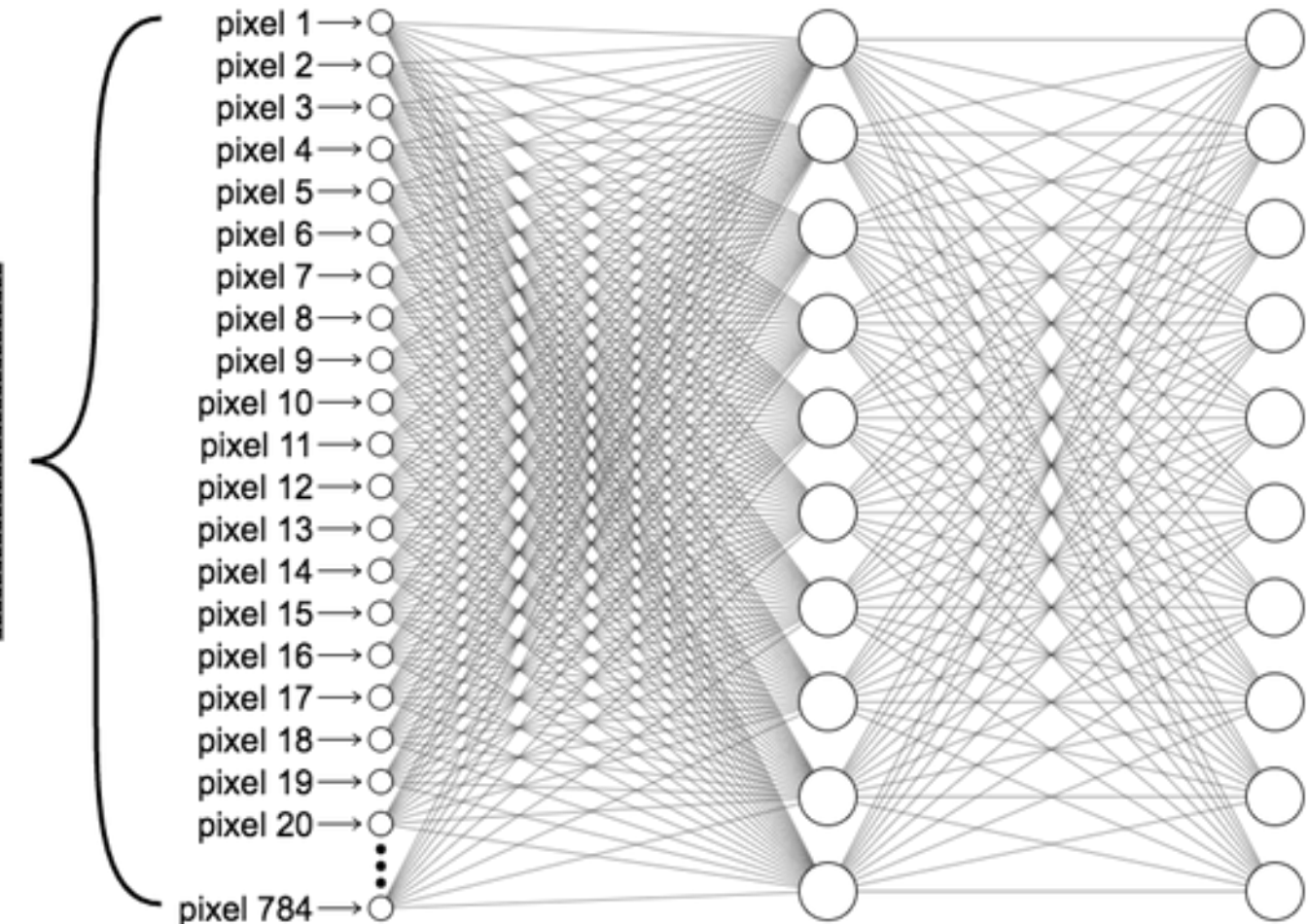
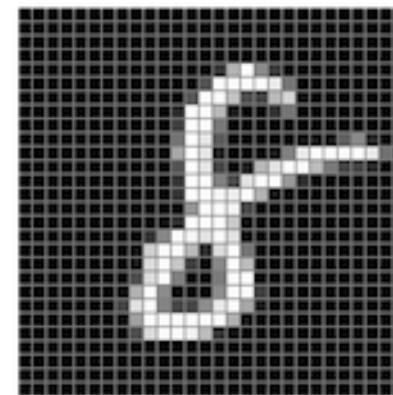
FEATURE EXTRACTION + CLASSIFICATION



Convolutional Neural Network

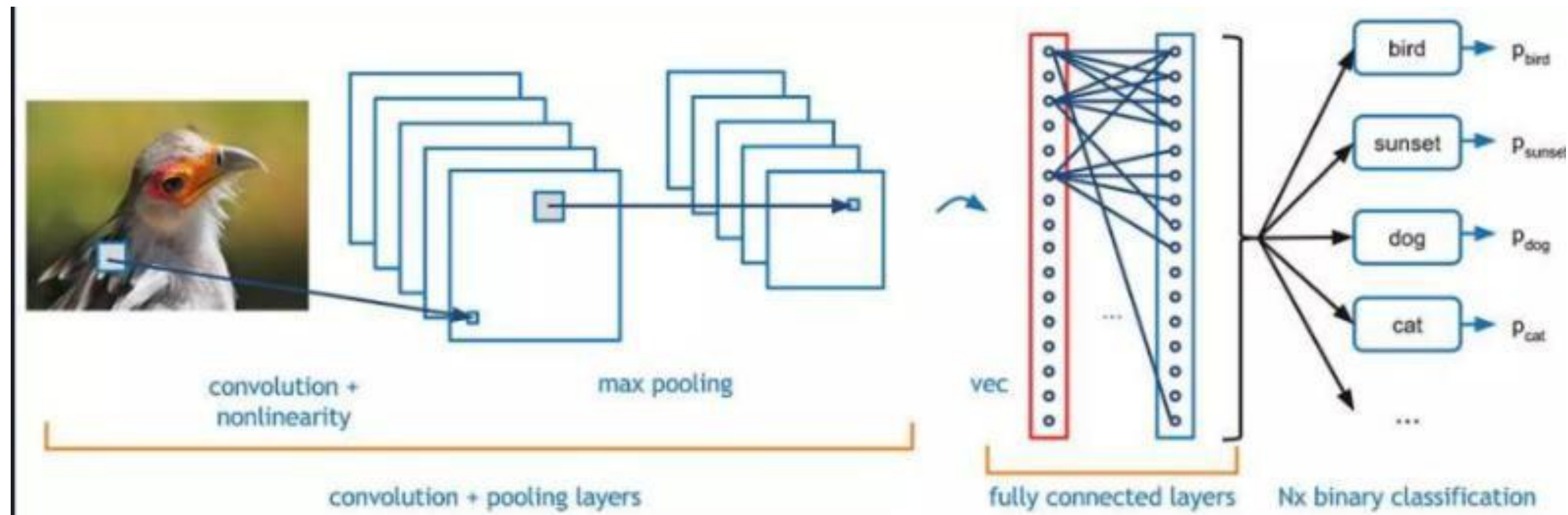


Contoh data MNIST



Convolutional Neural Network

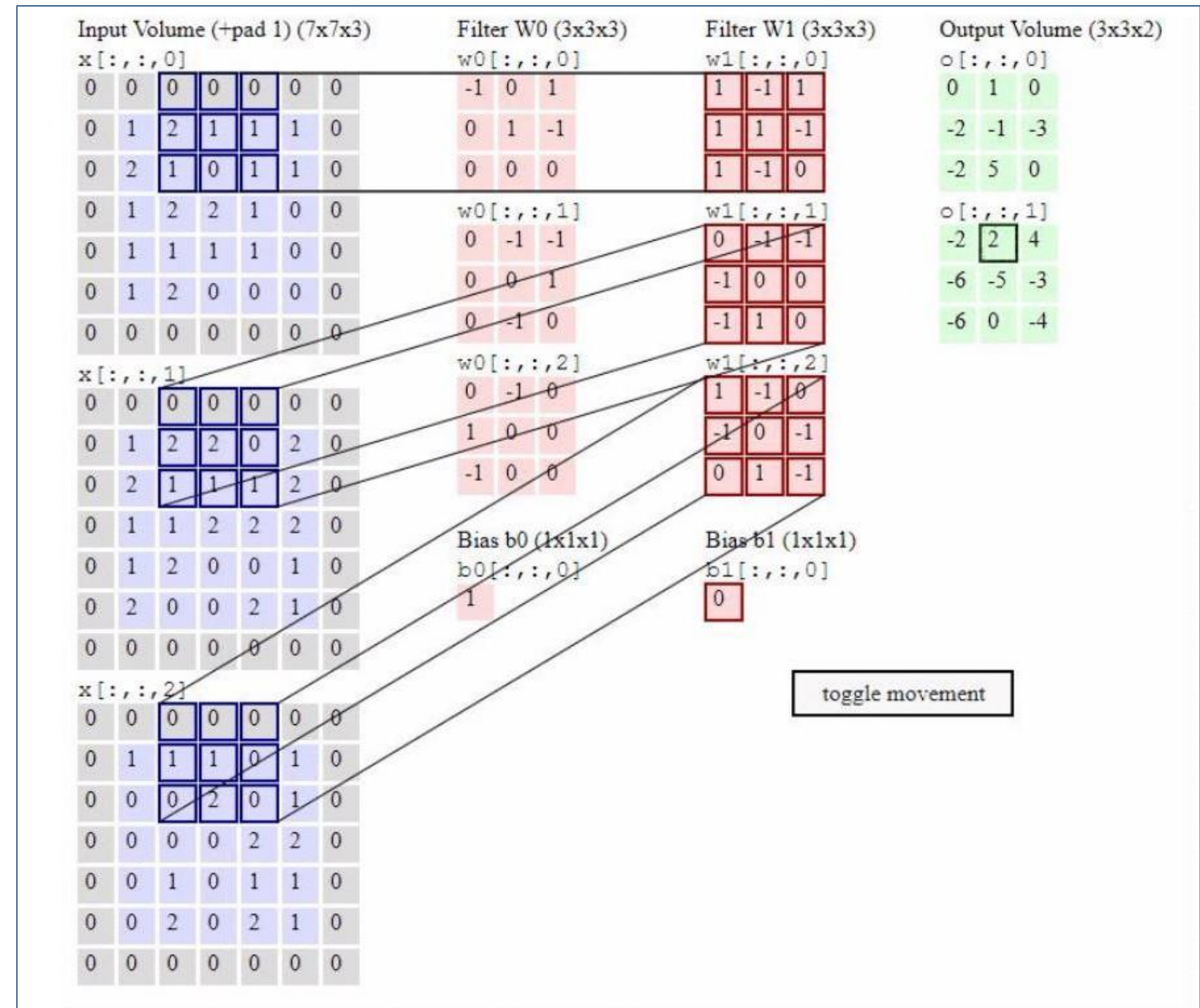
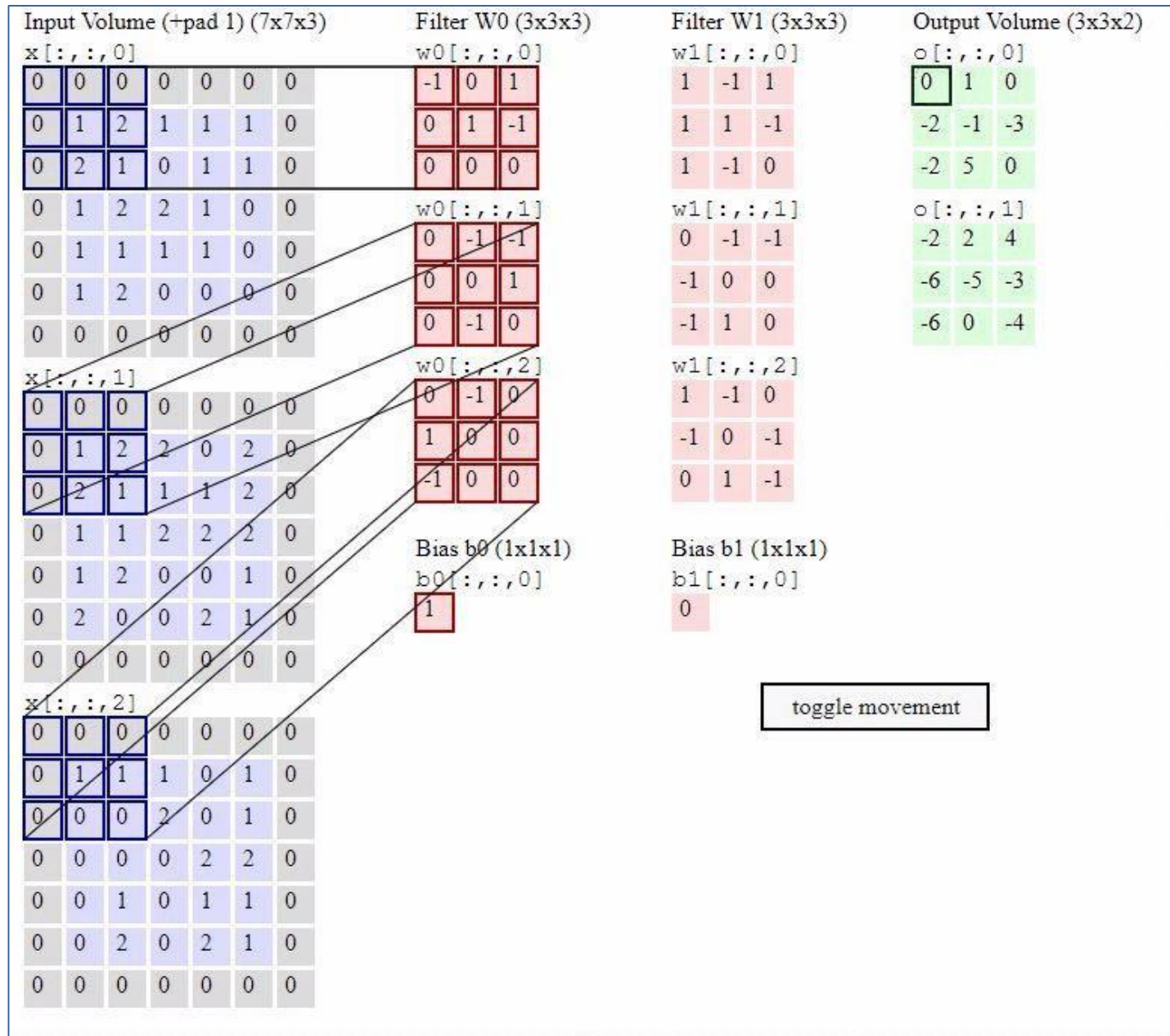
- **Jaringan Syaraf dengan operator konvolusi**
- Dirancang khusus untuk permasalahan dengan struktur seperti grid berdimensi rendah (seperti, gambar 2D)
- **Sparse representation** - tidak setiap neuron terhubung ke setiap neuron lainnya



Convolution Layer

- Komputer membaca gambar sebagai piksel dan merepresentasikannya sebagai matriks ($N \times N \times 3$) — (tinggi $\times$ lebar $\times$ kedalaman).
- Layer Konvolusi menggunakan serangkaian filter yang dapat dipelajari. Filter ini digunakan **untuk mendeteksi keberadaan fitur atau pola tertentu yang ada pada gambar asli (input) -> seperti tepi, garis dan lengkungan pada gambar.**
- Filter biasanya direpresentasikan sebagai matriks **3×3 atau 5×5** , dengan dimensi lebih kecil namun memiliki kedalaman yang sama dengan input.
- Filter ini digeser (convolve) ke seluruh lebar dan tinggi gambar input, lalu dilakukan operasi **dot product** untuk menghasilkan peta aktivasi (activation map).

Convolution Layer - Filter



Activation Function for Convolution

- Setelah hasil konvolusi, CNN menggunakan fungsi aktivasi seperti ReLu, sigmoid, dan tan.
- Jika menggunakan ReLu, maka fungsi ini akan mengubah nilai **negative menjadi nol**.

$$f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$$

Pooling Layer

- Pooling layer biasanya berada setelah conv. layer. Pada prinsipnya pooling layer terdiri dari sebuah filter dengan ukuran tertentu yang akan bergeser pada seluruh area feature map.
- Pooling yang biasa digunakan adalah **Max Pooling dan Average Pooling**.

Sebagai contoh jika kita menggunakan Max Pooling 2x2, maka pada setiap pergeseran filter, nilai maximum pada area 2x2 pixel tersebut yang akan dipilih, sedangkan Average Pooling akan memilih nilai rata-ratanya.

Fully-Connected Layer

Feature map yang dihasilkan dari feature extraction layer masih berbentuk multidimensional array, sehingga kita harus melakukan “flatten” atau reshape feature map menjadi sebuah vector agar bisa kita gunakan sebagai input dari fully-connected layer.

FC Layer yang dimaksud disini adalah **MLP yang sudah pernah kita pelajari sebelumnya**. FC Layer memiliki beberapa hidden layer, activation function, output layer dan loss function.



</> Implementasi — TensorFlow / Keras

1 Preprocessing

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
train_data = train_datagen
    .flow_from_directory(
        'data/train',
        target_size=(128,128),
        class_mode='categorical'
    )
```

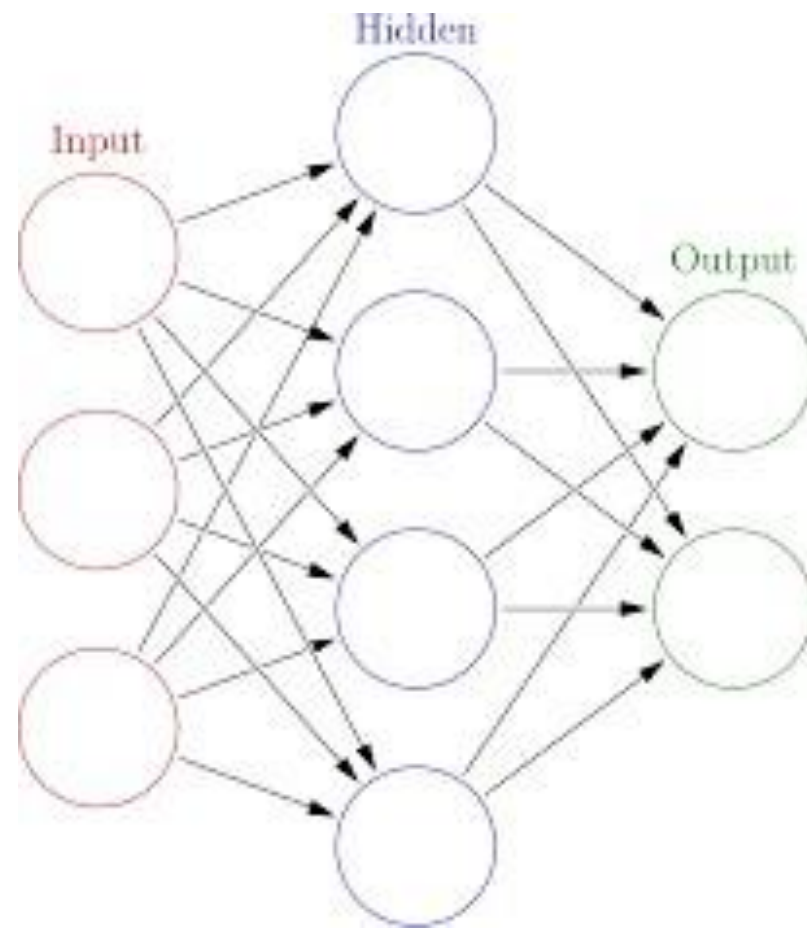
2 Building CNN Model

```
model = Sequential([
    Conv2D(32, (3,3),
        activation='relu',
        input_shape=(128,128,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(num_classes,
        activation='softmax')
])
```

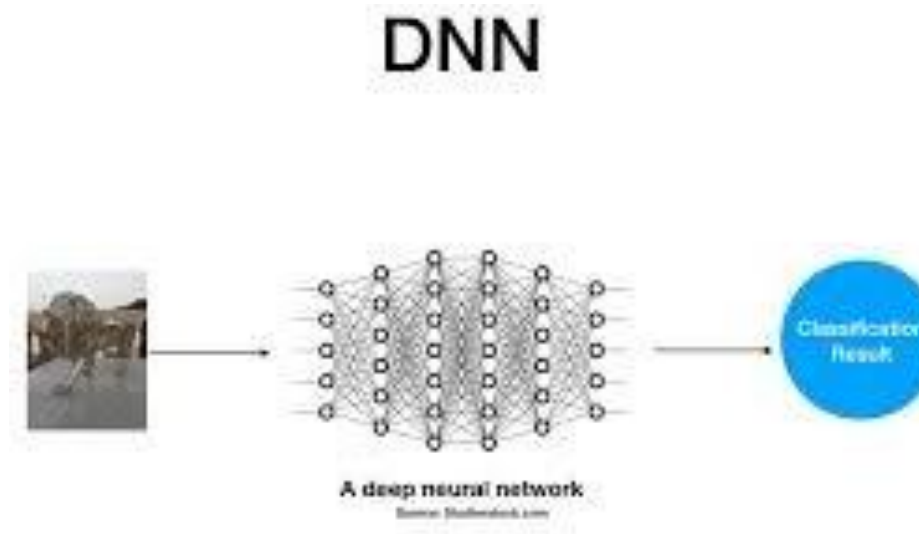
3 Compile & Training

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(train_data, epochs=10)
```

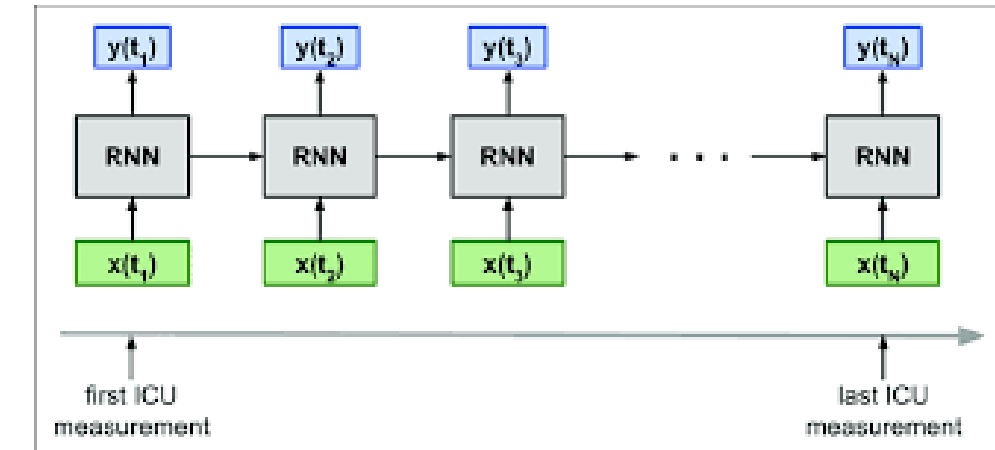
3 deep learning methods



ANN



CNN



RNN



ANN / MLP

Artificial Neural Network

- Input: fitur tabular (angka terstruktur)
- Fully-connected: setiap neuron terhubung ke semua neuron
- Cocok untuk klasifikasi dan regresi data tabular
- Tidak mempertahankan struktur spasial data



CNN

Convolutional Neural Network

- Input: gambar (data spasial 2D/3D)
- Sparse connections via filter konvolusi
- Weight sharing — parameter jauh lebih efisien
- Mempertahankan struktur spasial gambar



RNN

Recurrent Neural Network

- Input: data sekuensial (teks, suara, time series)
- Memiliki memory — output bergantung pada input sebelumnya
- LSTM/GRU mengatasi vanishing gradient
- Cocok untuk NLP dan prediksi deret waktu

Application Of Deep Learning Methods

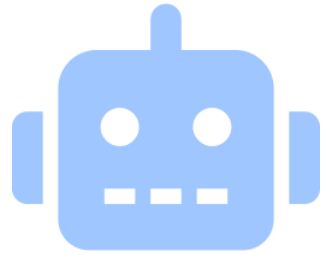


<https://compas.co.id>

Conclusion

Convolutional Layer bekerja dengan cara mengaplikasikan filter pada input melalui operasi konvolusi. Proses ini memungkinkan CNN untuk:

- ✓ Mendeteksi fitur lokal seperti tepi, sudut, dan tekstur
- ✓ Menggunakan parameter secara efisien melalui sharing
- ✓ Mempertahankan struktur spasial dari data input
- ✓ Membangun representasi hierarkis dari sederhana ke kompleks



P R A C T I C E

Image Classification

Deteksi Jenis Sampah di Sekitar Kampus

Dataset: <https://www.kaggle.com/datasets/farzadnekouei/trash-type-image-dataset>

Reference

1. <https://medium.com/@samuelsena/pengenalan-deep-learning-part-7-convolutional-neural-network-cnn-b003b477dc94>
2. <https://www.slideshare.net/slideshow/convolution-neural-network-cnn/135630477>

Preprocessing

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

train_data = train_datagen.flow_from_directory(
    'data/train',
    target_size=(128, 128),
    class_mode='categorical'
)
```

Model Training

```
model.fit(train_data, epochs=10)
```

Building CNN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(128, 128, 3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(train_data.num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

THANK YOU!